BANORTE

GUIA DE TRABAJO

GIT

PARA USO EXCLUSIVO DE

BANORTE SEGUROS Y PENSIONES

ELABORADO POR:

ARQUITECTURA TI

BANORTE

GIT

Contenido

1. ¿QUE ES GIT?... 3
2. Antes de iniciar 4
3. Configuración 4
4. Flujo de trabajo Git 5
5. Repositorio 6
6. Área de preparación 6
7. Agregar Cambios al Repositorio 7
8. Repositorios Remoto.....7
9. Ramas.....8
10. Clonar de Repositorio Remoto a Local 9
11. Actualizar y Funciones.....9
12. Remplazar cambios locales 10
13. Tags o Etiquetas.....11
14. Roll Back 11
15. Clonar de Local a Remoto.....12
16. Comando Blame.....14

1. ¿QUE ES GIT?

Git es un software de control de versiones diseñado por Linus Torvalds, pensando en la eficiencia, la confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones cuando estas tienen un gran número de archivos de código fuente

2. Antes de iniciar…

1 – Instalamos Descargando Git en nuestro equipo atreves del portal git con la liga :

https://gitforwindows.org/

3. Configuración:

Podemos configurar nuestro nombre y correo para identificar quien interactúa con los cambios y podamos referirlos a nosotros:

```
$ git config --global user.name "John Doe" $ git config --global user.email
“johndoe@example.com”
```

Colores especiales para la consola

```
git config color.ui true
```

Mostrar sólo una línea por cada commit en la traza

```
git config format.pretty oneline
```

Interfaz gráfica por defecto:

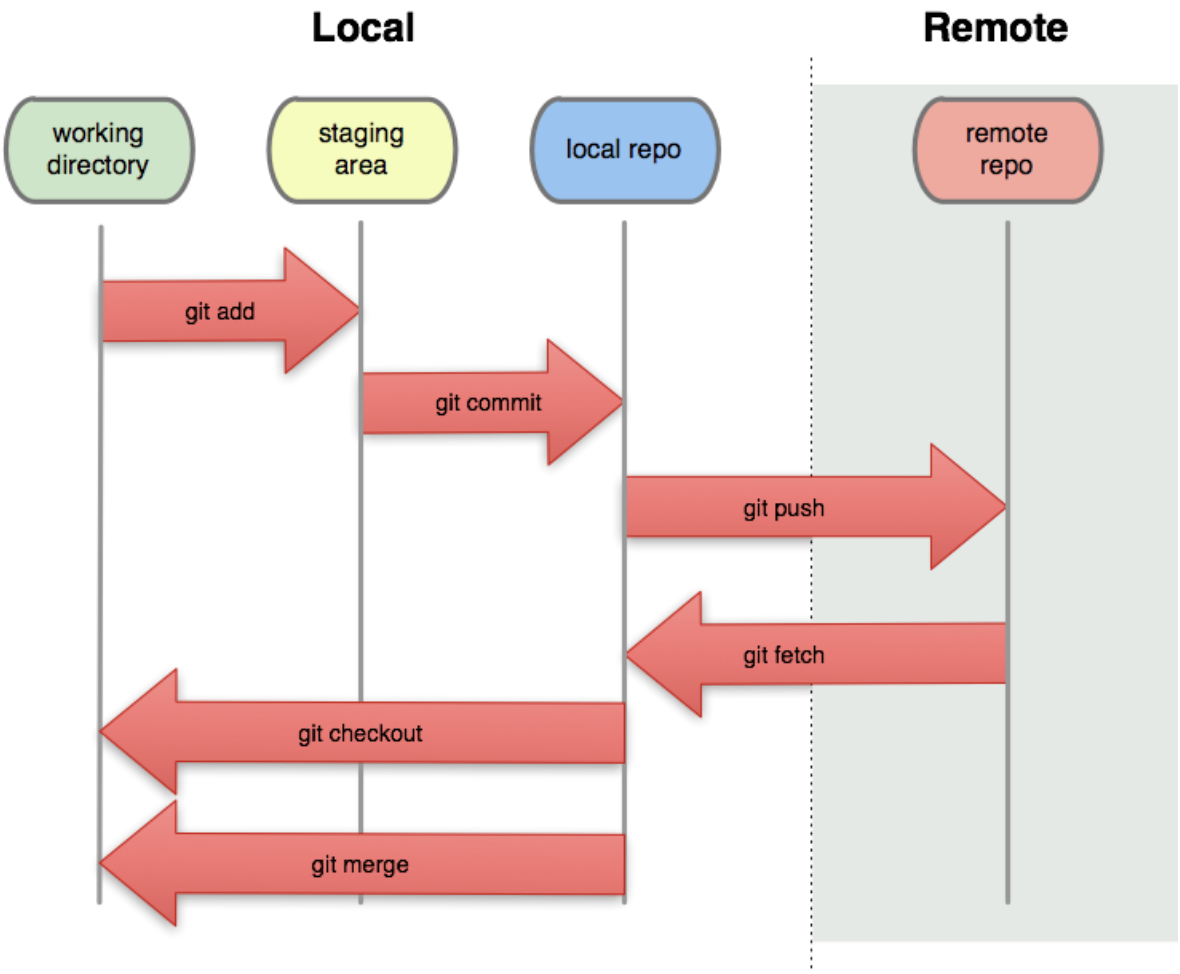
```
gitk
```

Agregar archivos de forma interactiva

```
git add -i
```

4. Flujo de trabajo Git

El flujo de trabajo en la transición de inicio a fin dentro de nuestro repositorio de modo que podamos sacar provecho a nuestro desarrollo de forma más eficiente y fluida



Podemos interpretarlo a través de 4 bloques

Nuestro directorio: La carpeta donde guardamos nuestros documentos a trabajar, una vez que los modificamos usamos el comando git add para agregar al área de preparación.

Staging Area o Area de preparación o índice: es un archivo de tu directorio Git, en él se encuentra qué información estará en el próximo git commit. A veces, nos referimos a él como el "índice".

Repositorio local: El repositorio es donde están los archivos donde trabajamos, es donde se guarda nuestra información y está lista para recuperar y/o subir a otro repositorio remoto
*Head: apunta al último commit realizado y genera/guarda una instantánea de nuestro trabajo.

Repositorio Remoto
Es aquel que no nos pertenece, el cual puede estar en un servidor en la nube, en el computador de otro desarrollador.

5. Repositorios

Creamos el repositorio en nuestra carpeta con el comando

```
$ git init
```

La carpeta que se creó con git init, tiene el nombre de ".git" donde guarda una instantánea del ultimo commit de nuestro proyecto.

Así se crea nuestra rama maestra o principal de forma automática en donde se encuentran nuestros documentos

6. Área de preparación

Agregamos nuestros cambios al staging área o área de preparación, este es el primer paso antes de colocar estos cambios en el repositorio:

```
$ git add -A
```

"-A" indica agregar todo (o git add <filename> para agregar un documento en específico)
NOTA: Usar "git status" para visualizar que si se agregaron los documentos

7. Agregar cambios a Repositorio

Agregamos nuestros cambios o nuevos documentos al repositorio local:

```
"git commit -m <nombre de la versión>"
```

Estos cambios están en el archivo HEAD (ve commit como una instantánea de los que se a echo guarda una imagen en el último punto de la rama en que trabajamos. Yo sugiero usar "tags" o etiquetas para poder hacer un git roll back a ese punto en específico si queremos regresar, esto aplica igual mente a otras ramas con más facilidad, podemos ver la lista de commits a través de nuestra rama de trabajo consultado $ git log -p)

8. Repositorios remotos

Usamos el comando:

```
$ git remote add origin <url del repositorio>
```

Esto agregara la dirección del repositorio remoto al que quieres conectarte para subir los cambios. Es probable que la primera vez te pida tu contraseña y usuario para poder conectarte.
De esta forma se agrega la dirección del repositorio web con el que queremos trabajar.

Ahora usamos:

```
$ git push origin master
```

para subir nuestros cambios, hay que recordar que podemos sustituir "master" por en nombre de la rama al que queremos hacer los cambios.

Nota: Así podrás subir tus cambios a cada rama correspondiente

9. Ramas

Espacios independientes dentro del proyecto, en conjunto son la base de trabajo en equipo, la idea es que cada individuo trabaje independiente mente en una rama generando sus propios cambios.

Estos cambios se fusionen con la rama principal ya sean grandes o pequeños
Creamos una nueva rama con

```
$ git checkout -b SubRama_Claus
```

Nota: podemos usar el mismo comando para navegar entre ramas ya existentes.

Seguimos nuestro flujo de trabajo agregando contenido en nuestro git

```
git add <filename>

git commit -m "nombre de la versión 1 de Claus
```

y agregamos nuestros cambios a nuestra rama

```
 $ git push origin SubRama_Claus
```

NOTA: Seleccionar otra rama solo es necesario remplaza master por el nombre de la rama que se requiera dirigir para trabajar "git push origin RamaCambiosClaus"

NOTA: Regresar a nuestra rama principal para trabajar vamos a "git checkout master"

NOTA: Borrar una rama "git branch -d RamaCambiosDescartados"

Ejemplo: "git push origin RamaCambiosClaus" o a remoto "git remote add origin <server o url>"

Una rama nueva no estará disponible para los demás a menos que subas (push) la rama a tu repositorio remoto

```
"git push origin <branch>"
```

10. Clonar de Repositorio remoto a Local

```
$ git fetch
```

Es un comando principal que se usa para descargar contenidos desde un repositorio remoto.
git fetch se usa en combinación con git remote, git branch, git checkout y git reset para actualizar un repositorio local al estado de un remoto.

```
$ git fetch <url del repositorio y rama>
```

11. Actualizar y fusionar

Para actualizar tu repositorio al último commit ejecutamos git pull en el directorio de trabajo donde nos encontramos trabajando
Así descargamos y fusionamos los últimos cambios

Para fusionar una rama en especial a tu rama activa (en la que nos encontramos, recuerda que podemos cambiar de rama con el comando $ git checkout -b SubRama_Ejemplo) usamos

```
git merge <rama>
```

No siempre será posible y se podrán producir conflictos. Tú eres responsable de fusionar esos conflictos manualmente al editar los archivos mostrados por git. Después de modificarlos, necesitas marcarlos como fusionados con:

```
"git add <filename>
```

Antes de fusionar los cambios, puedes revisarlos usando:

```
$ git diff <rama_que_quiero_fusionar_a_mi_rama> <rama_que_quiero que_se
actualize_que_se_encuentra_dentro_de mi_carpeta_detrabajo>
```

12. Reemplaza cambios a local

Puedes reemplazar cambios locales usando el comando:

```
 "git checkout -- <filename>"
```

Este comando reemplaza los cambios en tu directorio de trabajo con el último contenido de HEAD. Los cambios que ya han sido agregados al Index, así como también los nuevos archivos, se mantendrán sin cambio.

Por otro lado, si quieres deshacer todos los cambios locales y commits, puedes traer la última versión del servidor y apuntar a tu copia local principal de esta forma:

```
$ git fetch origin

$ git reset --hard origin/master
```

13. Tags o Etiquetas

Básicamente son etiquetas que adjuntamos a nuestros commits para identificar el cambio y poder referenciarnos a este si queremos retomar a partir de ese punto. Esto nos es útil más adelante para poder recuperar una versión a partir de un punto en específico si algún error se encontró en la última versión , para recuperar la última versión funcional

```
$ git tag -a v0.8 -m "versión 0.8 de nuestro proyecto del día 12/08/20XX,
donde Juanito agrego tal cambio"
```

NOTA: se usó v0.8 ya que es el commit 8

Hacemos $ git log para ver nuestros commits , copiamos el código sha que esta sobre la fila de author y date y colocamos :

```
 $ git tag -a v0.8 -m "versión 0.8 de nuestro proyecto del día 12/08/20XX,
donde Juanito agrego tal cambio" <código>
```

14. Roll Back

Si emos encontrados un error y deseamos regresar a un punto anterior en la rama principal o alterna consultamos los logs de la rama en la que estamos trabajando, seleccionamos el ultimo commit funcional en la rama y a través de crear una nueva podemos seguir adelante

Para volver a un commit anterior, primero busque el hash de la confirmación usando $ git log.
Para volver temporalmente a ese commit, separe el Head con:

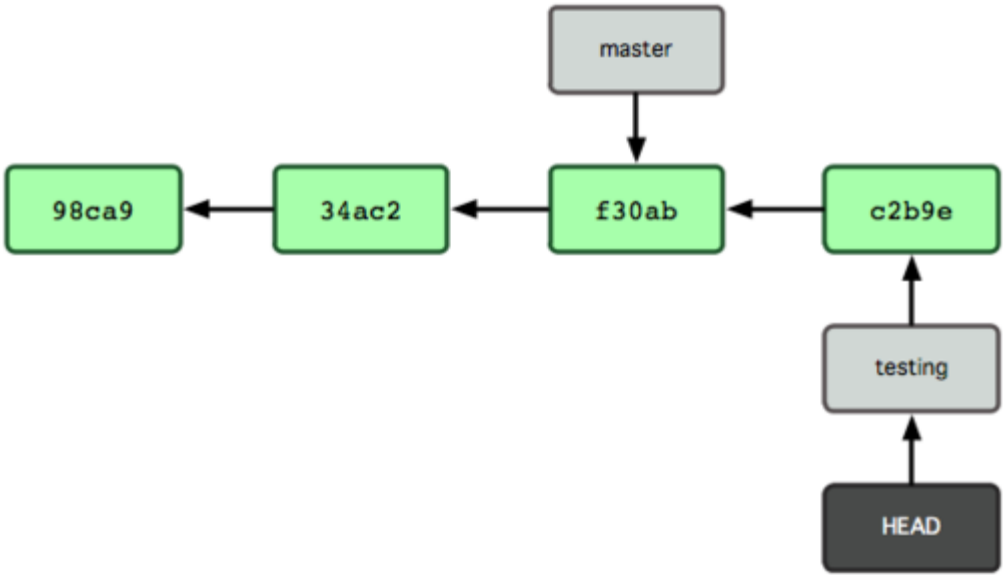
```
$ git checkout 789abcd
```

Esto nos coloca en el commit histórico 789abcd , ahora hacemos un "roll back" o regreso con:

```
$ git reset --hard 789abcd
```

O creamos una rama nueva a partir de ese punto con :

```
$ git checkout -b aNewBranch
```



15. Clonar de Local a Remoto

Ingresa a un repositorio remoto y copia la urlReposotorio.

Abrimos Git bash en una carpeta donde colocaremos nuestro repositorio.

Ejecutamos el siguiente comando para crear una copia local del repositorio ejecutado

```
$ git clone /path/to/repository
```

En un repositorio remoto:

```
$ git clone username@host:/path/to/repository
```

Hacer un clon local que comparta nuestra rama desde el repositorio actual:

```
$ git clone -l -s -n . ../copy

$ cd ../copy

$ git show-branch
```

Veamos los repositorios a los que estamos conectados.

```
$ Git remote -v
```

Consultamos las Branch existentes remotas y locales.

```
$ Git Branch -a
```

Descargar cambios del repositorio remoto y los actualizara en local.

```
$ git fetch origin branch-name
```

Descargara los cambios solo de la Branch y actualizara.

```
git fetch <remote_name> <branch_name>
```

Final mente podemos subimos los cambios/avances solo de la Branch al repositorio con la url de la rama que buscamos actualizar a la que tenemos permisos.

```
$ git push <remote>
```

16. Blame

El comando git blame sirve para examinar el contenido de un archivo línea a línea y ver cuándo se ha modificado y quién es el autor de las modificaciones.

```
$ git log --pretty=oneline -S'blame_usage'

5040f17eba15504bad66b14a645bddd9b015ebb7 blame -S <ancestry-file>

ea4c7f9bf69e781dd0cd88d2bccb2bf5cc15c9a7 git-blame: Make the output
```

Más Información Departamento de Arquitectura TI Autores de publicación:

Claudio Alonso Vargas claudio.vargas.organista@banorte.com
Martin Mesinos Villanueva martin.mesinos.villanueva@banorte.com
Alexis Jesus Rodriguez alexis.arrambide.rodriguez@banorte.com